

APV2: 面向持续认知的白箱预测-行动闭环架构

日期: 2026-06-14

稿件类型: 发布版论文稿

项目仓库: <https://github.com/ginsonko/Artificial-PsyArch-V2>

GL 开放中文对话仓库: <https://github.com/ginsonko/APV2-GL-OpenWorld-Chinese>

实验复现与冻结锚定仓库: <https://github.com/ginsonko/APV2-Reproduction-Artifacts>

第三方独立复现参考: https://github.com/ACG-j/artificial_psyarch

发布冻结 tag: apv2-release-20260614-final-longreport

复现锚点: 以公开仓库 tag、各仓库 PUBLIC_STAGING_MANIFEST.json 和外层 PUBLIC_REPO_STAGING_SUMMARY.json 为准。

配套长技术报告: APV2_长篇技术报告_完整论证与实验附录_20260614.docx / .pdf。本文是短主文, 负责给出问题、机制、关键证据和结论; 长技术报告承载完整定义、算法、运行 trace、实验附录、术语表和审稿疑问回应。

摘要

大语言模型已经证明了规模化语言建模的强大能力, 但持续认知不是把提示词写得更长, 也不是在模型外部挂接一个检索数据库。一个真正持续运行的心智系统需要拥有自己的当前状态场: 外界输入、刚发生的想法、历史经验、预测压力、认知感受、情绪慢量、行动倾向和行动后果, 都应在同一个循环中持续影响后续注意、回忆、预测和行动。

本文提出 APV2, 一种面向持续认知的白箱预测-行动闭环架构。APV2 使用统一的状态原子(state atom, SA)承载输入、行动、反馈、认知感受、短期叙事回读和记忆痕迹; 使用双能量状态池表达现实在场强度、预测强度及二者的认知压力; 使用 B/C/C* 组织相似状态召回和后继预测; 使用短期叙事槽维持有序的当前思维片段; 使用过程生成的认知感受和情绪慢量调制行动竞争; 使用行动反馈和持久化记忆闭合下一轮经验。

APV2 的重点不是替代大模型, 而是补足当前 AI 系统较少正面处理的一层: 一个系统如何在连续时间中始终作为同一个自己运行、学习、修正、恢复和复用经验。实验结果显示, APV2 底层循环在参数扰动、短期槽顺序消融、打断恢复、节奏后继、持久化重载、残差召回压力、压力动力学扫参、在线 learned vector

消融和第三方 Rust 复现中均有可观测证据。GL 开放中文对话线进一步在 teacher-off/no-leakage 约束下完成冻结题库与受控 Live Fresh300 验证，显示 AP-style 学习协议可组织出稳定的日常中文开放对话基础能力。

本文的贡献是提出并实现了一条与“更大模型、更长上下文”互补的路线：将持续认知拆成可运行、可观察、可消融、可复现的白箱动力学组件，并通过 AP-Core、GL 学习验证和第三方复现三条证据线共同支撑。

关键词: APV2; 持续认知; 白箱认知架构; 状态池; 短期叙事槽; 残差召回; 后继预测; 认知感受; 行动反馈; teacher-off 验证

Abstract

APV2 is a source-available research prototype for continuous cognition. Instead of treating memory as an external transcript or tool database, APV2 represents perception, short-term narrative state, historical recall, successor prediction, cognitive pressure, process-grounded feelings, action competition, and action feedback inside one auditable predictive-action loop. The architecture uses state atoms (SA), a dual-energy state pool, residual B recall, C/C* successor prediction, short-term narrative slots, emotion and cognitive-feeling modulation, local online learned vectors, and persistent memory writeback.

The current evidence package combines three lines of validation. AP-Core experiments test the runtime mechanisms directly, including parameter sensitivity, short-term-slot order ablation, interruption recovery, rhythm-shaped successor replay, persistence reload, residual-depth recall, pressure dynamics, and online-vector ablations. GL experiments test whether the same learning protocol can organize open Chinese dialogue behavior under teacher-off and no-leakage constraints, including Skill38 and OpenWorld-Foundation Fresh300 evaluations. A third-party Rust implementation further supports the portability of AP-inspired bounded mechanisms. Together, these results position APV2 as a reproducible engineering prototype for white-box continuous cognition, complementary to large language models rather than a replacement for them.

Keywords: APV2; continuous cognition; white-box cognitive architecture; state atom; residual recall; successor prediction; cognitive feelings; action feedback; teacher-off validation

主要贡献

本文贡献可以概括为四点。

1. 提出并实现 APV2 白箱预测-行动闭环，把状态、预测、短期叙事、感受、行动、反馈和记忆放入同一条持续 tick 循环。
2. 给出 AP-Core 机制证据，证明短期叙事槽、残差 B 召回、C/C* 后继预测、压力动力学、持久化重载和在线 learned vector 都可以被独立测试和消融。
3. 给出 GL 学习验证证据，证明 AP-style 学习协议可以在 teacher-off/no-leakage 条件下组织出稳定的中文开放对话基础能力。
4. 给出第三方独立复现证据，证明核心思想可以跨语言和工程路线重建，而不是只存在于单一代码库的内部实验中。

本文采用“短主文 + 长技术报告 + artifact 仓库”的发布结构。短主文保持可读性，长技术报告保留完整机制、公式、伪代码、tick trace、实验记录和边界讨论，artifact 仓库提供可复验文件、manifest 和 SHA-256 锚点。这样做的原因是 APV2 不是单一模型组件，而是一套持续认知架构；若把全部实现细节塞入短主文，会破坏阅读节奏，但若只保留短主文，又无法充分回答白箱性、可复现性和机制边界问题。

1. 引言

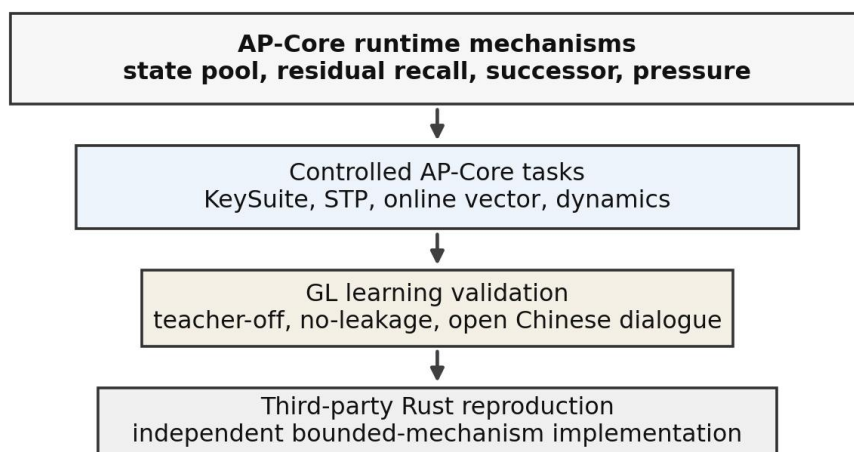
今天的大模型擅长语言生成、代码、问答、工具调用和多模态组织。它们可以在一次交互中表现得非常聪明，却不天然解决一个更底层的问题：系统怎样在连续时间中保持同一性。

普通 agent 可以保存聊天日志，也可以把工具调用结果写进数据库。但日志和数据库通常是外置材料：它们可能被检索回来，却不一定成为主体当前状态的一部分。一个持续认知系统需要的不只是“过去发生了什么”的记录，而是“过去如何以能量、压力、期待、感受和行动倾向继续留在现在”的机制。

APV2 研究的正是这一问题。它把感知、回忆、预测、行动、反馈和学习放进同一条 tick 循环。每一轮循环中，系统都把当前输入和内部回读转成 SA，放入状态池；由快系统做相似状态召回和后继预测；由注意选择形成短期叙事槽；由慢系统回看近期有序片段；由认知感受和情绪慢量调制行动；再把行动结果写回状态池和持久化记忆。

这条路线不否认大模型的价值。相反，大模型可以成为教师、工具、评测者和外部知识源。但 APV2 的学生侧能力不依赖答案表、关键词硬门、regex route、隐藏 solver、学生侧 LLM 或整句动作宏。能力来自可审计的状态-预测-行动-反馈-记忆闭环。

Figure 1. APV2 evidence stack



The layers support one release package but answer different claims.

图 1. APV2 证据分层全景

2. APV2 的核心对象

2.1 状态原子 SA

SA 是 APV2 的最小状态对象。文本、图像、音频、行动、反馈、认知感受、短期槽回读和控制状态都可以被编码为 SA。这样做的价值是统一：外部输入不是一类特殊指令，行动后果也不是旁路日志，它们都能进入同一个状态场参与竞争。

这带来两个关键结果。

第一，行动和反馈成为一等公民。一次说话、等待、修改、点击、失败、奖励或惩罚，都能成为未来相似状态下召回的材料。

第二，认知感受不来自外界字段改名。AP-native 的困惑、压力、不确定、错配、闭合、疲劳等感受只能由内部过程量生成，例如预测落差、证据缺口、短期重复、总虚能量变化率、行动失败压力等。外界文本只能作为普通 SA 进入状态池，不能直接变成内部感受。

2.2 双能量状态池

状态池是 APV2 的当前认知场。每个对象同时具有 real energy 和 virtual energy。real energy 表示当前真实在场、被观察或被行动反馈确认的强度; virtual energy 表示由记忆、预测、短期槽回读和期待形成的预测强度。二者的差异形成认知压力。

这不是装饰性的情绪标签，而是动作竞争中的变量。压力动力学实验显示，当认知压力升高时，系统不再继续把直接提交维持为主导，而会转向回看、替换和回放等更保守行动。进一步的压力 sweep 显示，clean 条件下 text_commit 保持正值但随压力单调下降，stress 条件下 text_commit 被压到 0，text_replace 和 replay_episode 随压力上升。这说明压力效应是可扫描复现的动力学曲线，而不是单点偶然。

2.3 快系统与慢系统

APV2 的快系统面向无序但有能量的状态场。它擅长快速召回“当前像过去什么状态”，并把相似历史的后继预测叠加回来。快系统不要求严格顺序，因此适合大量 SA 的并行粗召回。

慢系统面向短期叙事槽。短期槽保存最近若干 tick 的注意焦点包，每个槽不是单个 SA，而是一个 tick 内的多通道注意对象集合。槽内顺序只提供弱偏置，槽间相对顺序提供更强偏置，但都不是硬门。顺序正确会获得更高权重，顺序不完全匹配仍可被召回。

快系统让系统能迅速“想起类似情况”；慢系统让系统能维持“刚才在想什么”。二者叠加后，APV2 既有状态场式的快速联想，也有短期叙事式的连续思维。

2.4 残差式 B 召回

APV2 的 B 召回不是一次性取 top-k 后结束，而是逐轮吸收。每轮召回一个最强 B 对象，然后根据相似度降低当前 query 中已匹配 SA 的参与权重。下一轮召回自然转向剩余未被吸收的部分。

这个过程类似复杂信号经过共振结构：与结构 A 匹配的部分被吸收后，剩余信号中其他成分的相对重要性上升。实验中，混合 query 可以被逐轮分解出 AB、CD、E 等不同 winner，residual mass 逐轮下降。这使 APV2 能处理多对象、多行动、多线索同时存在的状态，而不是被第一个强匹配完全吞掉。

2.5 C/C* 后继预测与节奏

C 表示相似历史对象的后继预测，C* 表示多个后继预测叠加后的当前预测包。APV2 的后继偏置强调下一 tick 峰值：lag 1 最高，lag 2 断崖式下降，远端保留较低尾巴。这样可以支持诗句、儿歌、节奏动作和连续回放。

关键点是后继偏置不是固定 n-gram, 也不是答案表。它依赖当前状态池、短期槽、历史相似对象和后继波峰是否清晰。当没有清晰后继波峰时, 系统偏向模仿、回看或请求澄清; 当出现清晰后继波峰时, 系统偏向接着说、聚合或连续行动。

2.6 短期叙事槽作为内源性刺激包

短期叙事槽每个 tick 都会把自身内容回读成 `short_term_slot::* SA`, 以虚能量注入状态池。这相当于系统每一刻都把“刚才显性意识里亮着的东西”重新投回当前场, 维持思维连续性。

短期槽会生成 `summary`、`item`、`order`、`continuity`、`rhythm` 等通道。不同槽位有不同槽位系数, 越新的对象权重越高, 但强对象也可以跨 tick 保持。这样既能保留近期输入, 也能保留重要焦点。它不是外部输入, 而是叙事性内感受槽: 类似人类用短期记忆中的印象维持一个显性认知片段, 再据此判断、接话或行动。

2.7 核心过程的形式化摘要

为避免“白箱”停留在口号层面, APV2 在实现中把每个 tick 表达为可审计的状态变换。设状态原子集合为 $S_t = \{s_i\}$, 每个 s_i 带有实能量 $r_i(t)$ 、虚能量 $v_i(t)$ 和来源通道 `channel_i`。状态池更新可写成:

```
r_i(t+1) = decay_r * r_i(t) + sensory_i(t) + action_feedback_i(t)
v_i(t+1) = decay_v * v_i(t) + successor_i(t) + short_term_readback_i(t) + learned_vector_i(t)
pressure_i(t) = |v_i(t) - r_i(t)| * salience_i(t)
```

残差式 B 召回不是一次性 top-k。给定当前 query 质量 q_i^0 , 第 k 轮选择相似度最高的历史对象

B_k :

```
sim(B, q^k) = \sum_i q_i^k * match(s_i, B) / (\sum_i q_i^k + eps)
energy(B_k) = sim(B_k, q^k) * \sum_i (r_i(t) + v_i(t)) * q_i^k
q_i^{k+1} = q_i^k * (1 - absorb_rate * sim(B_k) * match(s_i, B_k))
```

这意味着已被 winner 解释的成分在下一轮自然降权, 未被解释的成分相对上升。C/C* 后继预测则把被召回对象的未来片段按 lag kernel 叠加:

```
C*(x) = \sum_k energy(B_k) * \sum_{lag} K(lag) * support(B_k, lag, x)
K(1) >> K(2) > K(3) ... > 0
```

短期叙事槽每 tick 把最近若干槽位回读为内源性 `short_term_slot::* SA`:

```
slot_energy(item) = base_readback * slot_coeff(slot_rank) * item_weight * continuity_gate
order_bias = weak_in_slot_order + stronger_cross_slot_relative_order
```

认知感受来自过程量，而不是外界字段改名。例如，预测熵升高和 C* 峰不清晰会提高低把握/困惑; Σ pressure_i 和行动失败压力会提高回看、替换和请求澄清的 drive; 奖励/惩罚只通过行动后果写入后续记忆，不倒灌成当前行动前的证据。

表 1 给出短主文中的核心公式如何落到当前实现字段。完整参数表、代码路径和真实 JSON 摘录见配套长技术报告的“期刊最终版附录”。

机制	公式/变量	代码字段与默认值	可观测 trace
双能量状态池	$r_i(t+1), v_i(t+1)$	StatePoolConfig.real_decay=0.9, virtual_decay=0.86	state_snapshot.items[*].real_energy / virtual_energy / cognitive_pressure
固定预算读出	$S_t \rightarrow \text{bounded } R_{\text{state}}$	r_state_head_limit=7, r_state_items_per_head=256, memory_snapshot_limit=1024	r_state_snapshot/v1.heads, memory state_field_items
注意选择	focus_score	focus_limit=8, pressure_gain=0.6, virtual_energy_weight=0.25	focus.selected_labels, attention ranked rows
短期叙事槽	slot_energy(item)	capacity=32, base_virtual_budget=0.72, item_rank_decay=0.86, item_order_decay=0.92	short_term_slot::summary/item/order/continuity/rhythm
残差 B 召回	q_i^{k+1}	recall_top_k=5, scoring_candidate_limit=96, residual_recall_deepen_grasp_threshold=1.01	residual_b_recall_round/v1
C/C* 后继预测	$C^*(x), K(\text{lag})$	predict_top_k=5, prediction_energy_scale=0.55, successor_bias_gain=0.42	memory.fast_cn / slow_cn, cstar_budget_trace/v1
在线 learned vector	learned_vector_i(t)	dim=32, learned_weight=0.28, per_tick_update_limit=8	vector_spaces.hash_vector / online_learned_vector

2.8 白箱 tick trace 示例

下面先给出一个压缩示意，再给出真实 JSON 摘录位置。假设用户输入“关关雎鸠”，系统历史中存在诗句后继经验：

tick	状态池主峰	短期叙事槽	B 召回/残差	C* 后继峰	行动倾向
t0	text::关, text::关, text::雎, text::鸠 实能量上升	槽 0 保存当前注意包	第 1 轮 winner 解释“关关雎鸠”片段，残差质量下降	text::在 lag1 峰清晰	接着说 drive 上升
t1	short_term_slot::	槽 0 为“关关雎	第 2 轮转向未解	text::在 胜出，	输出 在

	关关雎鸠 虚能量回读	鸠”，槽间连续性高	释后继成分	text::河 为下一步候选	
t2	输出反馈把 text:: 在 写回状态池	槽 0 更新为 “关关雎鸠，在”	已匹配 “在” 的 query 成分降权	text::河 成为 lag1 峰	输出 河 或继续回放

如果此时外界打断或 C* 没有清晰唯一峰，注意力会转向新输入、回看或请求澄清，而不是继续按固定 n-gram 输出。这个例子说明 APV2 的“接着说”来自状态池、短期槽、残差召回和 C* 后继峰共同作用，不是答案表或整句动作宏。

真实记录摘录已固定在 `paper_artifacts/release_20260614/apv2_real_trace_excerpt_20260615.json`。该文件从 `outputs/apv21_debug_fresh10_turn9_fulltrace/sessions/dlg-a6a07b0851414743.json` 和 `outputs/apv22_apcore_dynamics_20260614_022059/apv22_apcore_dynamics.json` 抽取，记录原始 SHA-256，并保留真实字段名。一个代表性 tick 摘录如下：

```
{
  "tick_index": 201,
  "input_text": "我有点累的时候，你应该先怎么回应？",
  "focus": {
    "selected_labels": [
      "feeling::coherence",
      "action_feedback::stabilize_prediction",
      "timefelt::elapsed"
    ]
  },
  "feelings": {
    "grasp": 0.8478,
    "pressure": 0.375,
    "uncertainty": 0.3742,
    "evidence_gap": 0.2474
  },
  "selected_actions_top4": [
    {"action_id": "action::stabilize_prediction", "drive": 1.8},
    {"action_id": "action::recall_recent_context", "drive": 1.1734},
    {"action_id": "action::continue_focus", "drive": 0.9368},
    {"action_id": "action::wait", "drive": 0.9217}
  ],
  "short_term_slot_items_preview": [
    "short_term_slot::summary",
    "short_term_slot::item::feeling::coherence",
    "short_term_slot::item::action_feedback::stabilize_prediction",
    "short_term_slot::order::0::feeling::coherence"
  ]
}
```

AP-Core dynamics 摘录同时记录了残差召回的逐轮吸收、后继 lag kernel 和压力 sweep。例如 ResidualDepth-1 中前三轮 residual mass 从 4.6 -> 2.1671 -> 1.3533 -> 0.7742,

SuccessorPeakAblation-1 中 lag kernel 为 1.0, 0.42, 0.1101, 这些都来自机器可读 JSON, 而不是手写图解。

3. 学习机制: 从模仿到开放对话

APV2 的语言学习路线不是关键词路由或答案表, 而是一个分阶段学习过程:

- 5. echo imitation: 听到并复述局部声音或字形。
- 6. successor prediction: 从已见片段中预测下一个对象。
- 7. multi-reply aggregation: 多个可能回复叠加, 形成候选表达族。
- 8. process-paradigm binding: 把内源性认知状态和表达范式绑定, 例如困惑时更容易学习 “为什么” “这是什么” 。
- 9. keyword organization: 词汇和主题开始组织经验邻域, 但不成为硬门。
- 10. grammar/style refinement: 在经验、反馈和教师示范中形成更自然的语法与风格。

GL 线负责把这套学习协议变成可执行课程、teacher-off 测试和 no-leakage 审计。LLM 在这里可以作为外部成人教师或事后阅卷者: 它可以示范、补课、生成题目、事后评分, 但不能在学生侧替 AP 作答。学生侧必须保持无 provider、无 student-side LLM、无答案表、无 regex route、无整句动作宏、无 hidden solver。

在开放中文对话验证中, GL 已经覆盖日常问候、情绪回应、未知问题边界、澄清请求、桌面协助、重复输入、噪声输入、记忆回看、简单数学、教学请求等类别。OpenWorld-Foundation 受控 Live Fresh300 记录为 300/300, reply unique 为 294, ablation 为 6/6; Skill38 Codex Fresh300 teacher-off 冻结题库记录为 300/300, no-leakage 为 300/300, 题库 SHA-256 为 c55e39e307092295568e417e01f17978aaf297748054674d9b7b03bc6ab35f5e。这说明 AP-style 学习协议可以在严格审计下组织出稳定、可解释、可复验的中文开放对话基础能力。

4. 实验证据

4.1 AP-Core 机制证据

AP-Core 证据回答的是底层循环是否按设计运行。

证据	结果	支持的结论
APV2-BottomLoop-ParamSensitivity-1	16/16 pass	底层循环不是单点参数偶然

ShortTermSlot-OrderAblation-1	full-order margin 18.2466, without-order margin 9.0539	顺序是软偏置，不是硬门
LongRun-InterruptionRecovery-1	interruptions 2, resumptions 2, final slot virtual mass 1.3715	短期叙事可在受控打断后恢复
RhythmSuccessor-Replay-1	lag 1 为 1.0, lag 2 为 0.42, lag 4 为 0.172	后继预测具有下一拍峰和衰减尾巴
PersistenceBackend-Reload-1	warm-load loaded 3, JSONL SHA-256 recorded	记忆可以跨本地文件边界重载
ResidualDepth-Stress-1	8 winners, 7 residual rounds, mass 14.3126 -> 0.7448	残差 B 召回在混合 query 下仍可解释
ShortTermSlot-Grid-1	108/108 pass	短期槽容量和能量预算有界可控
DoubleEnergyBalance-PressureDynamics-1	stress 下 text_commit 近零, replace/replay 上升	压力会重排行动竞争
DoubleEnergyBalance-PressureDynamics-Sweep-1	clean/stress sweep 均呈稳定趋势	压力效应是可扫参曲线

4.2 在线 learned vector 证据

APV2 的在线 learned vector 是辅助分支，不取代 SA、B/C/C* 或状态池定义。它的作用是把共现、共同聚焦、反馈和认知压学习到一个本地、可审计、有界的经验邻域。

三项消融显示:

证据	结果	支持的结论
OnlineVector-WeightAblation-1	neighbor score 0.6848 -> 1.0460 -> 1.6308; exact match 始终 rank 0	learned vector 有用但不越界
OnlineVector-NegativePressure-1	wrong residue 0.3598 -> -0.5937; correct association 只下降 0.0286	错误预测残留被定向清除
TransitionIsolation-1	learned transition A->B 0.0 -> 0.9412; concept similarity 0.1840 -> 0.1840	后继学习不污染概念相似

4.3 GL 开放中文对话验证

GL 证据回答的是学习协议是否能把 AP 底座组织成可用的中文对话基础能力。它与 AP-Core 机制证据分层，但二者互相支撑: AP-Core 提供可解释底座，GL 提供教学、验证和开放对话课程。

证据	结果	支持的结论
OpenWorld-Foundation Controlled Live Fresh300	300/300, average score 100.0, reply unique 294, ablation 6/6	日常开放中文对话场景下, teacher-off/no-leakage 验证稳定通过
Skill38 Codex Fresh300 Teacher-Off Run	300/300, threshold >=290, no-leakage 300/300	冻结题库上对话范式、边界反应和后验评分均通过
Skill38 readiness/cold retest	domain 6/6, cold retest 6/6, ablation degradation 6/6	学习阶段、冷保持和机制消融可观察

DailyDialogueSkill 系列	多轮 random300 / natural semantic / high-confidence 验证	基础日常中文对话课程已形成可扩展技能包
-----------------------	--	---------------------

4.4 与真实 LLM/agent 的受控对照

在 RepeatMap v0.5 / LongRun 受控任务中，AP-style 内生反馈闭环与真实 LLM/agent 路线进行了同台对照。G3 使用真实 Claude 无记忆，G4 使用真实 GPT 加 memory/tool。结果显示：

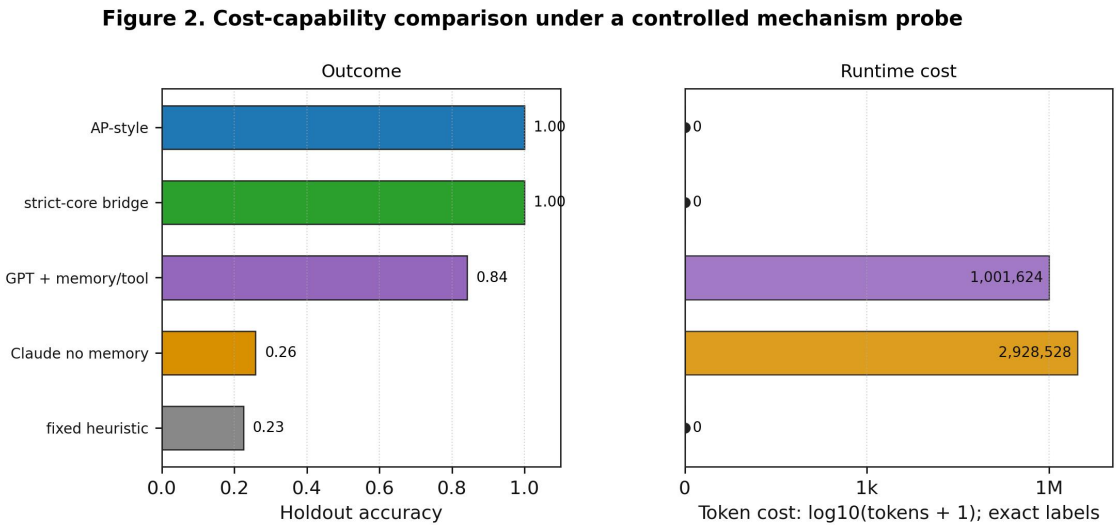


图 2. 受控任务中的成本-能力对照

路线	alpha holdout	beta holdout	API calls	tokens	tools
G1A AP-style	1.0000±0.0000	1.0000±0.0000	0	0	0
G1B strict_core bridge	1.0000±0.0000	1.0000±0.0000	0	0	0
G2 fixed heuristic	0.2000±0.2400	0.2500±0.2191	0	0	0
G3 Claude no-memory	0.2167±0.0400	0.3000±0.0980	717	2,928,528	0
G4 GPT+memory/tool	0.8333±0.2066	0.8500±0.1819	707	1,001,624	660

这组结果的意义不是简单宣称 AP 全面超过大模型，而是说明机制差异：在需要跨轮反馈学习、重载保持和规则切换再学习的受控任务中，AP-style 内生记忆闭环可以以零 token、零外部 API 展示可审计学习；强 LLM-agent 路线也有效，但依赖外部调用、工具调度和 token 成本。

4.5 第三方独立复现

第三方作者在 ACG-j/artificial_psycharch 中使用 Rust 独立实现 AP-inspired bounded mechanisms, 并完成本地复跑。复跑结果包括：

- cargo check PASS;

- `cargo fmt --check` PASS;
- `cargo clippy ... -D warnings` PASS;
- `cargo test --lib 84/84` PASS;
- core report commands PASS;
- generated math train/holdout/generalization/reload accuracy 1.00;
- relation-word teacher-off/cold-retest/reload/holdout accuracy 1.0;
- control probes 区分 geometry-only、no-geometry 和 shuffled expected。

这条证据说明 AP-style 学习与审计结构可以跨语言、跨工程路线重建。它不是原仓库内部测试的重复，而是外部实现对核心原理可迁移性的支持。

5. 为什么 APV2 有独特价值

APV2 的独特性不在于它现在已经比所有大模型更强，而在于它把持续认知中最难说清的部分变成了可运行对象。

第一，它把“当前我在想什么”工程化。短期叙事槽让最近注意对象持续以虚能量回读入池，使思维连续性不只依赖长上下文文本。

第二，它把“我为什么犹豫、为什么回看、为什么修正”工程化。认知压力、错配、证据缺口和行动后果都能进入状态池和记忆，而不是事后解释。

第三，它把“学会”拆成可审计过程。不是直接给答案，而是通过观察、模仿、后继预测、范式绑定、反馈修正和冷保持逐步形成能力。

第四，它允许大模型成为教师而不是学生脑内的隐藏求解器。LLM 可以在提交后评估、示范和补课，但学生侧仍保持无 provider、无隐藏 solver、无整句宏。

第五，它天然适合长程产品形态。桌宠、桌面助手、学习伙伴和开放世界 agent 都需要持续状态、边界感、恢复能力、学习能力和可解释行为。APV2 提供的是这些能力的底座，而不是一次性回答器。

6. 仓库与复现

发布版采用三仓库结构：

11. Artificial-PsyArch-V2: APV2 core runtime、论文主文、AP-Core 机制实验、图表和基础复现说明。

12. APV2-GL-OpenWorld-Chinese: GL 学习协议、开放中文对话课程、Fresh300/Skill38 teacher-off 验证、no-leakage 审计材料。
13. APV2-Reproduction-Artifacts: 冻结实验产物、manifest、hash、第三方复现整理、复跑命令和 release 级 artifact。

第三方复现仓库 ACG-j/artificial_psyarch 作为独立参考链接保留。这样做可以让读者分清三件事: AP-Core 是机制底座, GL 是教学与验证层, artifact 仓库是复现实验锚点。

发布锚点采用三层记录, 避免把 zip hash 写入会被打包进 zip 的内部文件而造成递归漂移:

层级	锚定内容	审查用途
公开仓库 tag	apv2-release-20260614-final-longreport	固定 GitHub 可浏览源代码和文档版本
每仓库 manifest	PUBLIC_STAGING_MANIFEST.json	记录仓库内每个公开文件的 bytes 和 SHA-256
外层 release summary	release_repos_20260614/PUBLIC_REPO_STAGING_SUMMARY.json	记录三个外发 zip 包的 SHA-256 和字节数

许可证采用 APV2 Public Research License v2026-06-14: 允许公开阅读、clone、本地运行、非商业研究复验和学术引用, 同时保留商业使用、模型训练、数据再打包、产品部署和派生系统公开分发等权限边界。

7. 数据、代码与复现可用性

APV2 发布版采用三个公开仓库和一个统一 tag 固定代码、文档与实验产物。Artificial-PsyArch-V2 承载 AP-Core runtime、机制实验、短主文、长篇技术报告和图表; APV2-GL-OpenWorld-Chinese 承载 GL 学习协议、开放中文对话课程、teacher-off/no-leakage 验证和技能包材料; APV2-Reproduction-Artifacts 承载冻结 artifact、manifest、SHA-256、复跑命令和第三方复现整理。第三方 Rust 复现仓库 ACG-j/artificial_psyarch 作为独立参考实现保留。

复现锚点由三层组成: 公开仓库 tag apv2-release-20260614-final-longreport, 各仓库内的 PUBLIC_STAGING_MANIFEST.json, 以及外层 PUBLIC_REPO_STAGING_SUMMARY.json 中记录的 zip SHA-256。发布文档本身也由 paper_artifacts/release_20260614/release_manifest_20260614.json 记录 Word/PDF 文件的 bytes 与 SHA-256。这样读者可以从论文主张追溯到仓库文件、实验输出、文档版本和外发压缩包。

代码以 APV2 Public Research License v2026-06-14 发布为 source-available public research preview。该许可证允许公开阅读、克隆、本地复验、非商业研究评估和合理引用，同时保留商业使用、模型训练或评测数据复用、数据集/技能包再打包、产品部署和官方衍生版本声明等权限边界。

8. 适用边界与下一步

APV2 当前最扎实的结论集中在三个层面。第一，AP-Core 已经把状态池、短期叙事槽、残差召回、后继预测、认知压力、行动反馈、持久化和在线 learned vector 做成可运行、可观察、可消融的底层循环。第二，GL 学习验证表明，这套底座可以被教学协议组织成稳定的中文开放对话基础反应，并在 teacher-off、no-leakage、cold/retest 和 ablation 条件下接受复验。第三，第三方 Rust 复现说明核心机制具有跨工程路线迁移的可能性。

APV2 与拥有海量世界知识和强自然语言生成能力的大模型形成互补。面对百科事实、复杂数学竞赛、长篇创作、代码工程或多工具开放规划等任务，APV2 可以通过教师、工具、外部知识源和更长课程扩展能力；它当前最突出的优势在另一层：把当前状态、预测落差、短期叙事、行动反馈和学习痕迹组织成可审计的持续认知场。也就是说，APV2 是一个可教学、可回看、可修正的认知底座；LLM 则适合承担教师、外部知识源、解释器和工具编排者。二者的关系是互补，而不是单纯胜负。

这使 APV2 更适合被理解为持续认知架构的可复验工程原型，区别于一次性问答模型或产品包装。下一步最有价值的推进方向包括：更长时间的真实在线运行、更多领域的 teacher-off/cold retest、真实桌面/桌宠长期交互、更多第三方独立实现、正式化引用与 venue 模板，以及把 artifact 仓库中的冻结证据扩展成更标准的学术复现包。这样的路线可以把当前的机制证明、学习验证和外部复现进一步推进为长期开放环境中的稳定能力证据，并为读者讨论 APV2 在开放世界智能路线中的位置提供更扎实的材料。

9. 结论

APV2 给出了一条与主流大模型路线互补的认知工程路线。它不把智能只看成一次输出，而把持续运行中的状态、预测、压力、注意、记忆、行动和反馈组织成同一条白箱循环。AP-Core 机制实验说明这条循环可以被参数扰动、顺序消融、打断恢复、节奏回放、持久化重载、残差召回、压力动力学和在线 learned vector 逐项检查。GL 开放中文对话验证说明这套底座可以被教学协议组织成稳定的日常对话基础能力。第三方 Rust 复现进一步说明核心机制不绑定单一代码库。

因此，APV2 的核心成果可以概括为：它把“一个系统如何持续成为它自己”从哲学口号推进为可运行、可审计、可教学、可复现的工程原型。

10. 参考文献

[LeCun2015] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521, 436-444. DOI: 10.1038/nature14539.

[Vaswani2017] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems* 30. arXiv:1706.03762.

[Brown2020] Brown, T. B., Mann, B., Ryder, N., et al. (2020). Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems* 33.

[Yao2023ReAct] Yao, S., Zhao, J., Yu, D., et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations*. arXiv:2210.03629.

[Shinn2023Reflexion] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366.

[Park2023GenerativeAgents] Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative Agents: Interactive Simulacra of Human Behavior. *UIST 2023*. arXiv:2304.03442.

[Newell1990] Newell, A. (1990). *Unified Theories of Cognition*. Harvard University Press.

[Laird2012Soar] Laird, J. E. (2012). *The Soar Cognitive Architecture*. MIT Press.

[Anderson2004] Anderson, J. R., Bothell, D., Byrne, M. D., Douglass, S., Lebiere, C., & Qin, Y. (2004). An Integrated Theory of the Mind. *Psychological Review*, 111(4), 1036-1060. DOI: 10.1037/0033-295X.111.4.1036.

[Friston2010] Friston, K. (2010). The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11, 127-138. DOI: 10.1038/nrn2787.

[Clark2013] Clark, A. (2013). Whatever next? Predictive brains, situated agents, and the future of cognitive science. *Behavioral and Brain Sciences*, 36, 181-204. DOI: 10.1017/S0140525X12000477.

[Pineau2021] Pineau, J., Vincent-Lamarre, P., Sinha, K., Lariviere, V., Beygelzimer, A., d'Alche-Buc, F., Fox, E., & Larochelle, H. (2021). Improving Reproducibility in Machine Learning Research. *Journal of Machine Learning Research*, 22(164), 1-20.